Lecture 03: Assignment 2 Problem Set & Systems Case Studies

Welcome to the official problem specification catalog for **Lecture 03: Python Fundamentals (Part 2 — Conditionals, Loops & Functions)**. This document contains formal problem definitions, mathematical formulations, algorithmic constraints, and test vectors for all 10 Assignment 2 problems alongside 3 systems engineering case studies.

📋 Table of Contents

- Problem 01: Progressive Income Tax Rate Calculator
- Problem 02: Inclusive Range Even Number Stream
- Problem 03: High-to-Low Digit Deconstruction
- Problem 04: Integer Digit Cardinality Counter
- Problem 05: Arithmetic Sum of Integer Digits
- Problem 06: Dual Divisibility Filter in Fixed Interval
- Problem 07: Sentinel-Controlled Polling Loop
- Problem 08: Multi-Way Arithmetic Dispatch Calculator
- Problem 09: Sub-Linear Primality Verifier
- Problem 10: Interactive Number Guessing Engine
- Systems Case Study 1: Control Flow Branching & Bytecode Jump Offsets
- Systems Case Study 2: Loop Execution Speed & VM Interpreter Overhead
- Systems Case Study 3: Call Stack Frame Allocation & Recursion Limits

---

Problem 01: Progressive Income Tax Rate Calculator

Problem Description

Write a Python program that accepts an individual's salary as input. Using structured conditional statements (`if` , `elif` , `else`), compute and display the applicable final tax percentage and total tax liability based on the following statutory rules:

- If $\text{salary} < 30,000 \implies 5\%$
- If $30,000 \leq \text{salary} \leq 70,000 \implies 15\%$
- If $\text{salary} > 70,000 \implies 25\%$

Mathematical Formulation

Let $S \in \mathbb{R}_{\geq 0}$ represent salary. The tax rate piecewise function $R(S)$ is:

$$R(S) = \begin{cases} 0.05 & \text{if } 0 \leq S < 30,000 \\ 0.15 & \text{if } 30,000 \leq S \leq 70,000 \\ 0.25 & \text{if } S > 70,000 \end{cases}$$

Total tax liability is computed as:

$$T(S) = S \cdot R(S)$$

Requirements & Constraints

- Validate that salary is non-negative ($S \geq 0$).
- Output the tax percentage and computed monetary tax formatted to two decimal places.
- Time Complexity Target: $\mathcal{O}(1)$.
- Space Complexity Target: $\mathcal{O}(1)$.

Problem 02: Inclusive Range Even Number Stream

Problem Description

Write a function `print_even_numbers(a: int, b: int) -> None` that accepts two integer boundaries a and b (where $a \leq b$) and prints all even integers located within the closed interval $[a, b]$ (inclusive).

Mathematical Formulation

Given integer domain $D = \{x \in \mathbb{Z} \mid a \leq x \leq b\}$, identify the subset:

$$E = \{x \in D \mid x \equiv 0 \pmod 2\}$$

Requirements & Constraints

- Support both $a \leq b$ and $a > b$ (normalize boundaries automatically).
- Print values space-separated or comma-separated on a single line.
- Time Complexity Target: $\mathcal{O}(|b - a|)$.
- Auxiliary Space Target: $\mathcal{O}(1)$.

Problem 03: High-to-Low Digit Deconstruction

Problem Description

Write a function that prints the constituent digits of a positive integer N in standard left-to-right positional order. For example, given $N = 312$, the function must output `3`, `1`, and `2`.

Mathematical Formulation

An integer N in base-10 positional notation has expansion:

$$N = \sum_{i=0}^{k-1} d_i \cdot 10^i, \quad d_i \in \{0, 1, \ldots, 9\}$$

The least significant digit is extracted via:

$$d_0 = N \pmod{10}$$

The truncated integer is obtained via:

$$N' = \lfloor N/10 \rfloor$$

Requirements & Constraints

- Implement using arithmetic successive division without converting the integer to a string (`str(N)`).
- Handle single-digit numbers ($N = 0$) correctly.
- Time Complexity Target: $\mathcal{O}(\log_{10} N)$.
- Space Complexity Target: $\mathcal{O}(\log_{10} N)$ for order reversal.

Problem 04: Integer Digit Cardinality Counter

Problem Description

Write a function `count_digits(n: int) -> int` that computes and returns the total number of digits contained within an integer N.

Mathematical Formulation

For an integer $N > 0$, the total number of decimal digits K satisfies:

$$K = \lfloor \log_{10} N \rfloor + 1$$

For $N = 0$, $K = 1$. For negative integers, $K(N) = K(|N|)$.

Requirements & Constraints

- Implement an iterative while loop using $N //= 10$.
- Handle $N = 0$ and negative integers without errors.
- Time Complexity Target: $\mathcal{O}(\log_{10} |N|)$.
- Space Complexity Target: $\mathcal{O}(1)$.

Problem 05: Arithmetic Sum of Integer Digits

Problem Description

Write a function `sum_of_digits(n: int) -> int` that calculates and returns the arithmetic sum of all decimal digits in a given integer N.

Mathematical Formulation

Given integer N with decimal digits $(d_0, d_1, \ldots, d_{k-1})$:

$$\Sigma(N) = \sum_{i=0}^{k-1} d_i = \sum_{i=0}^{k-1} \left(\lfloor |N| / 10^i \rfloor \pmod{10} \right)$$

Requirements & Constraints

- Input $N = 312 \implies 3 + 1 + 2 = 6$.
- Input $N = 9045 \implies 9 + 0 + 4 + 5 = 18$.
- Time Complexity Target: $\mathcal{O}(\log_{10} |N|)$.
- Space Complexity Target: $\mathcal{O}(1)$.

Problem 06: Dual Divisibility Filter in Fixed Interval

Problem Description

Write a program that prints all integers in the closed range $[1, 100]$ that are simultaneously divisible by both 3 and 5.

Mathematical Formulation

A number x is divisible by both 3 and 5 if and only if it is divisible by their Least Common Multiple (LCM):

$$3 \mid x \land 5 \mid x \iff \text{lcm}(3, 5) \mid x \iff 15 \mid x$$

The target sequence in range $[1, 100]$ is:

$$\mathcal{S} = \{x \in [1, 100] \mid x \equiv 0 \pmod{15}\} = \{15, 30, 45, 60, 75, 90\}$$

Requirements & Constraints

- Demonstrate both a brute-force loop testing `x % 3 == 0 and x % 5 == 0` and an optimized step-iteration loop `range(15, 101, 15)`.

Problem 07: Sentinel-Controlled Polling Loop

Problem Description

Design an interactive console program that continuously prompts the user to enter a number. For each input, report whether the number is **positive**, **negative**, or **zero**. The program must continue looping indefinitely until the user inputs the sentinel token `"Quit"` (case-insensitive).

Mathematical Formulation

Let input stream be sequence $(I_1, I_2, \ldots, I_m)$ where $I_m = \text{"Quit"}$. For each $k < m$:

$$\text{Classify}(x) = \begin{cases} \text{"Positive"} & \text{if } x > 0 \\ \text{"Negative"} & \text{if } x < 0 \\ \text{"Zero"} & \text{if } x = 0 \end{cases}$$

Requirements & Constraints

- Normalize user input using `.strip().lower()`.
- Guard against invalid non-numeric inputs using `try ... except ValueError`.
- Gracefully exit loop when sentinel `"quit"` is encountered.

Problem 08: Multi-Way Arithmetic Dispatch Calculator

Problem Description

Create a production arithmetic calculator function `calculator(a: float, b: float, operation: str) -> float` that performs addition, subtraction, multiplication, or division based on the `operation` parameter (`'+'`, `'-'`, `'*'`, `'/'`). Implement the multi-way dispatch using Python 3.10+ `match ... case`.

Mathematical Formulation

The binary arithmetic operator map is:

$$f(a, b, \text{op}) = \begin{cases} a + b & \text{if op} =' +' \\ a - b & \text{if op} =' -' \\ a \times b & \text{if op} =' *' \\ a / b & \text{if op} =' /' \wedge b \neq 0 \end{cases}$$

Requirements & Constraints

- Raise `ZeroDivisionError` or return custom descriptive error when $b = 0$ under division.
- Handle unsupported operator tokens with a helpful `ValueError`.
- Time Complexity Target: $\mathcal{O}(1)$.

Problem 09: Sub-Linear Primality Verifier

Problem Description

Write a function `is_prime(n: int) -> bool` that determines whether a given integer N is a prime number using a loop. The function must return `True` if N is prime, and `False` otherwise.

Mathematical Formulation

An integer $N > 1$ is prime if its only positive divisors are 1 and N. If N is composite, it must have at least one prime divisor d satisfying:

$$d \leq \lfloor \sqrt{N} \rfloor$$

Therefore, trial division needs only inspect potential divisors up to $\sqrt{N}$.

Requirements & Constraints

- Handle boundary values: $N \leq 1$ are NOT prime.
- Smallest prime is 2.
- Optimize trial division to skip even numbers after testing 2.
- Time Complexity Target: $\mathcal{O}(\sqrt{N})$.
- Space Complexity Target: $\mathcal{O}(1)$.

Problem 10: Interactive Number Guessing Engine

Problem Description

Create an interactive "Number Guessing Game". The program selects or stores a secret integer $S \in [1, 100]$. The user is repeatedly prompted to guess the number. On each guess:

- Print `"Too high"` if the guess exceeds S.
- Print `"Too low"` if the guess is below S.
- Print `"Correct!"` and terminate if the guess equals S.

Mathematical Formulation

Under binary search optimal play, the user can identify secret $S \in [1, 100]$ in at most:

$$K = \lceil \log_2 100 \rceil = 7 \text{ guesses}$$

Requirements & Constraints

- Maintain a guess counter and report total attempts upon success.
- Sanitize user input against non-integer strings.

Systems Case Study 1: Control Flow Branching & Bytecode Jump Offsets

Problem Statement

Disassemble Python's `if/elif/else` and `match/case` constructs using the standard library `dis` module. Analyze how CPython evaluates conditionals at the bytecode level via opcodes `POP_JUMP_IF_FALSE`, `JUMP_FORWARD`, and `COMPARE_OP`. Formulate the control flow graph.

Systems Case Study 2: Loop Execution Speed & VM Interpreter Overhead

Problem Statement

Benchmark the execution speed of `while` loops, `for` loops, and C-accelerated primitives (`sum(range())`) in CPython. Explain why `while` loops exhibit significant virtual machine interpreter overhead due to continuous variable rebinding and comparison opcodes compared to `FOR_ITER`.

Systems Case Study 3: Call Stack Frame Allocation & Recursion Limits

Problem Statement

Examine CPython's execution stack during deep recursion. Analyze `PyFrameObject` memory footprints, measure the call depth boundary, and demonstrate why CPython imposes `sys.getrecursionlimit() = 1000` to prevent C stack overflows.